IOT401 · EXAMEN

Maison connectée et supervision IoT sécurisée

Cas étudié : domotique - priorité à la vie privée.

Enseignant : Marouane El Moufid | **Établissement :** École 2600.

BINÔME	IDENTITÉ
Membre 1	Benjamin TIBI (GitHub : bentibi)
Membre 2	Ilias EL HOUZI (GitHub : SaitamaDuBled)

Quatre capteurs simulés (présence, température salon, chambre, cuisine), un broker MQTT local et un superviseur. Authentification réciproque par certificats, contrôle d'accès, segmentation Docker, audit et conformité.

État du dossier : infrastructure exécutée sur deux PC Ubuntu. Mesures et tests d'audit réussis sur le PC de Benjamin le 28/10/2026, preuves brutes enregistrées dans preuves/. Relecture et signature des contributions encore nécessaires.

Version : 28 octobre 2026 | **Rendu :** dossier PDF + archive des sources + preuves réelles.

1. Cahier des charges (CDC)

Contexte et priorité de sécurité

L'objectif est de superviser une maison sans transmettre les données d'occupation à un fournisseur cloud. Le capteur de présence peut révéler les heures de passage ou d'absence des habitants : notre priorité est donc la **confidentialité** et la minimisation des données. La détection de surchauffe complète cet objectif.

Objets du prototype

CN / certificat client	Mesure	Topic MQTT	Énergie envisagée
presence-entree	booléen oui/non	maison/presence-entree/etat	pile
temp-salon	température °C	maison/temp-salon/temperature	pile
temp-chambre	température °C	maison/temp-chambre/temperature	pile
temp-cuisine	température °C	maison/temp-cuisine/temperature	secteur

Contraintes du domaine

Portée : logement de quelques dizaines de mètres. Les murs peuvent atténuer le Wi-Fi et Zigbee. **Énergie :** limiter la fréquence des capteurs sur pile ; ici une simulation publie toutes les 8 à 10 s pour la démonstration. **Échelle :** quatre objets aujourd'hui, ajout de nouvelles identités/ACL à prévoir. **Réseau :** une box domestique peut héberger des objets compromis ; ne pas faire confiance au LAN. **Réglementation :** RGPD lorsque des données concernent des personnes identifiables et principes IoT de sécurité grand public.

Données envoyées

Chaque objet envoie uniquement `{"valeur": ...}` sur son topic dédié. Aucun nom, image, audio ou identifiant d'occupant. La valeur de présence est booléenne et non un historique de déplacements.

1. Cahier des charges (suite)

Choix réseau et applicatif

Technologie	Avantages	Limites / choix
Wi-Fi + TCP/IP	déjà présent dans les logements, débit confortable	consommation plus forte pour les piles ; adapté à la passerelle
Zigbee	faible consommation et maillage en intérieur	nécessite une passerelle Zigbee → IP/MQTT
Bluetooth LE	basse consommation, peu coûteux	portée et passerelle à considérer
MQTT v5 / TLS	léger, publication/abonnement, QoS et retours d'autorisation	retenu comme protocole applicatif du prototype
HTTP / HTTPS	simple pour API et intégration web	surdimensionné pour de courtes publications fréquentes

Décision de conception

Pour une maison réelle, des thermomètres sur pile pourraient utiliser Zigbee vers une passerelle locale, puis MQTT/TLS jusqu'au broker. Le capteur de cuisine alimenté sur secteur pourrait aussi utiliser le Wi-Fi. Dans **notre prototype**, les capteurs sont des processus Python reliés par TCP/IP sur un réseau Docker privé. Le lien radio n'est pas émulé : c'est une limite assumée.

MQTT : pourquoi ?

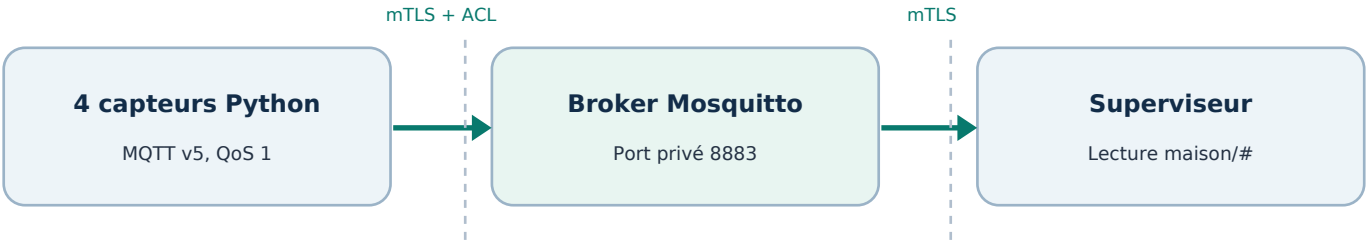
L'architecture est orientée événement. Les capteurs publient dans leur sujet, le broker distribue et le superviseur s'abonne. **QoS 1** donne un accusé de réception pour les démonstrations d'autorisation. Les messages ne sont pas conservés (`retain=False`) pour limiter les traces de présence.

Politique fonctionnelle

Le superviseur signale une **température strictement supérieure à 30 °C**. C'est un seuil pédagogique de surchauffe, non une certification de sécurité incendie. Il signale également tout topic de la maison qui n'appartient pas à l'ensemble des quatre sujets autorisés.

2. Architecture et limites de confiance

Chaîne de bout en bout et zones de confiance



Zone A : sources	Frontière de sécurité	Zone B : traitement local
4 capteurs Python : présence-entree temp-salon temp-chambre temp-cuisine	**mTLS** : chaque client prouve son identité ; le client vérifie le nom DNS et la CA du serveur. **ACL** : autorisation par CN et par topic.	Mosquitto sur 8883, privé au réseau Docker. Puis superviseur Python autorisé à lire `maison/#`.

Flux	Origine → destination	Protection / fonction
F1 (montant)	capteur → broker	publication QoS 1 sur son topic, mTLS, ACL
F2 (montant)	broker → superviseur	abonnement `maison/#`, mTLS, lecture seule
F3 (descendant)	superviseur → capteur	aucune commande descendante dans le prototype
F4 (gestion)	initialisation PKI → volumes Docker	certificats créés sans réseau, volumes par identité

Limites de confiance

La frontière principale sépare chaque capteur du broker : une adresse IP ou un nom de conteneur ne suffit pas à être reconnu. Le broker exige une preuve cryptographique et décide quels topics sont autorisés. Une deuxième frontière limite le superviseur à la lecture. La CA privée n'est montée que dans le conteneur d'initialisation et jamais chez les capteurs.

Edge plutôt que cloud

Le broker et le superviseur restent sur le PC local. Aucun serveur SaaS n'obtient les mesures. Il n'existe aucun port MQTT publié sur le réseau de l'hôte. L'hôte Docker et les administrateurs restent toutefois des points de confiance forts.

3. Sécurité implémentée

Identité des objets et provisioning

Au premier `docker compose up`, `initialisation-pki` crée une autorité locale, le certificat du broker et un certificat propre à chacun des quatre capteurs et au superviseur. Le CN (Common Name) identifie l'objet. Une identité `capteur-test` existe exclusivement pour la preuve d'intrusion du laboratoire.

Chaque couple clé/certificat client occupe **un volume Docker distinct**. Le broker ne reçoit que sa clé serveur et le certificat public de la CA ; les capteurs ne reçoivent que leur propre identité et la CA publique. La clé de signature de la CA est stockée dans un volume distinct non monté chez les capteurs.

Authentification mTLS et chiffrement

Contrôle	Implémentation vérifiable
Broker exige le client	`require_certificate true` dans `mosquitto.conf`
Nom MQTT = identité certifiée	`use_identity_as_username true`
CA contrôlée côté client	`ssl.create_default_context(... ca.crt)`
Nom du serveur validé	`check_hostname=True`, SAN DNS = broker
TLS minimal	TLS 1.2 minimum, sans option tls_insecure

Secret et stockage au repos

Le dépôt et l'archive ne contiennent **aucune clé privée**. Les clés du laboratoire sont des fichiers non chiffrés dans les volumes locaux, protégés par permissions et séparation des montages : cela **ne remplace pas** un composant matériel sécurisé ni le chiffrement du disque. En production : TPM/élément sécurisé, chiffrement des volumes, contrôle des accès administrateurs.

Durée des certificats

Les certificats de clients sont valides 90 jours et la CA 365 jours. Le script réutilise une identité existante : il n'effectue **pas** une rotation automatique avant expiration.

3. Sécurité implémentée (suite)

Moindre privilège : ACL appliquées par Mosquitto

Identité certifiée	Droit MQTT
presence-entree	écriture : maison/presence-entree/etat
temp-salon	écriture : maison/temp-salon/temperature
temp-chambre	écriture : maison/temp-chambre/temperature
temp-cuisine	écriture : maison/temp-cuisine/temperature
superviseur	lecture : maison/# ; pas d'écriture
capteur-test (audit)	écriture : maison/capteur-test/etat ; identité hors allowlist

Segmentation

Dans `compose.yaml`, le broker et ses clients utilisent le réseau Docker `maison-privee` déclaré `internal: true`. Aucun `ports:` ne publie 8883 vers le PC. Le conteneur de création des certificats utilise `network_mode: none`. Une compromission d'un conteneur du même réseau reste un risque résiduel.

Détection locale

Le superviseur vérifie le topic dans une liste blanche stricte de quatre sujets. Le cas d'audit `capteur-test` peut publier sur son propre topic, mais génère **ALERTE INTRUSION** car il n'est pas dans la liste blanche. Les températures supérieures à 30 °C génèrent **ALERTE SEUIL** ; les formats invalides provoquent **ALERTE FORMAT**. L'alerte est visible dans les journaux, sans application web.

Cycle de vie / intégrité : mesures prévues, non déployées

En production : délivrance de certificats après enrôlement contrôlé, renouvellement régulier, liste de révocation (CRL) ou OCSP, mises à jour OTA signées et vérifiées, démarrage sécurisé (secure boot), anti-rollback, inventaire des versions. Le prototype ne fournit ni firmware ni mise à jour OTA et la réinitialisation `docker compose down -v` révoque globalement les identités de laboratoire en régénérant la CA.

4. Rapport d’audit de notre infrastructure

Méthode STRIDE sur les frontières de confiance

Périmètre : flux F1 (capteur → broker), F2 (broker → superviseur) et F4 (provisionnement des clés). L’audit cible principalement l’observation illicite de la présence des habitants.

STRIDE	Scénario sur la chaîne	Contrôle / résiduel
S - Usurpation	Un faux capteur se présente comme temp-salon sur F1.	mTLS avec certificat dédié et CN vérifié ; vol de clé toujours possible.
T - Altération	Modification d’une température ou d’un message de présence en transit.	TLS garantit l’intégrité du flux ; capteur compromis hors périmètre.
R - Répudiation	L’émetteur nie une publication problématique.	Connexions et alertes journalisées ; pas de signature des données bout en bout.
I - Divulgation	Un autre capteur s’abonne à maison/# pour connaître l’occupation.	ACL : le superviseur seul peut lire ; trafic chiffré.
D - Déni de service	Messages excessifs empêchant la supervision.	Segmentation et faible nombre d’objets ; pas de quota anti-DoS applicatif.
E - Élévation	Un client tente d’écrire sur le topic d’un autre objet.	ACL explicites, test MQTT v5 : refus RC:135 attendu.

Hypothèses de l’audit

L’attaquant contrôle un client situé sur le réseau local du laboratoire mais ne dispose pas, par défaut, de la CA privée. L’administrateur du PC Docker reste une autorité de confiance ; l’isolation de conteneurs ne permet pas de résister à un administrateur hostile de l’hôte.

4. Rapport d’audit (menaces priorisées)

Arbre d’attaque : apprendre quand le logement est occupé

But : lire la présence en temps réel	Voies étudiées (OU logique)
A. Écouter le réseau	A1 : intercepter MQTT → chiffrement TLS ; A2 : usurper le broker → validation du nom et de la CA.
B. Abuser du broker	B1 : se connecter sans certificat → mTLS ; B2 : certificat d’un autre capteur et abonnement → ACL.
C. Voler des secrets / journaux	C1 : accéder au volume de clés ; C2 : administrer l’hôte Docker ; C3 : lire les logs → risques résiduels.

Priorisation et traçabilité (impact x vraisemblance, de 1 à 3)

Menace	I x V	Niveau	Contre-mesure dans le code
Lecture des présences	3 x 2	6 / élevé	ACL + mTLS ; mosquitto/acl.conf et config broker
Usurpation d’objet	3 x 2	6 / élevé	Certificat client unique ; app/pki.py
Vol de la clé CA	3 x 1	3 / moyen	Volume CA isolé ; compose.yaml
Message altéré en transit	2 x 2	4 / moyen	TLS + vérification du pair ; app/commun.py
Capteur inconnu dans la chaîne	2 x 2	4 / moyen	Allowlist et alerte ; app/superviseur.py
Déni de service	2 x 2	4 / moyen	Réseau interne ; quotas non implémentés

Résultat de l’analyse

Les risques prioritaires sont la divulgation des habitudes et l’usurpation d’objet. Les barrières mTLS, TLS, ACL et allowlist se complètent : une identité techniquement authentifiée ne doit pas suffire à obtenir l’accès à tous les messages.

4. Rapport d’audit (tests exécutés)

Tests réalisés le 28 octobre 2026 sur le PC Ubuntu de Benjamin. Résultats recopiés à partir des sorties réelles placées dans preuves/. Le code de retour et les alertes ont été vérifiés sur les logs du superviseur.

Vérification	Commande / source	Résultat observé
Chaîne complète	docker compose ps -a ; preuve-chaine.txt	Broker, superviseur et 4 capteurs démarrés. Mesures des quatre topics reçues.
Sans certificat client	audit-sans-certificat ; preuve-audit-sans-certificat.txt	Connexion rejetée par le broker : mTLS actif.
ACL d’un autre topic	audit-acl ; preuve-audit-acl.txt	Publication refusée : **RC:135** (Not authorized).
Objet hors liste blanche	audit-intrusion ; preuve-alertes.txt	Publication d’essai RC:0, puis **ALERTE INTRUSION** sur capteur-test.
Température > 30 °C	audit-seuil ; preuve-alertes.txt	Publication RC:0, puis **ALERTE SEUIL** à 35,5 °C.
Tests Python	unittest discover ; preuve-tests-unitaires.txt	**5 tests réussis sur 5.**
Réseau interne	compose.yaml ; preuve-infrastructure.txt	Réseau Docker internal: true et aucun port publié.

Problème rencontré et correction

Au premier lancement, Docker refusait la configuration des journaux : compression incompatible avec max-file = 1. La correction a été ajoutée par Ilias dans compose.yaml (max-file = 2, compression désactivée) et poussée sur GitHub. Après récupération du commit, le démarrage et les tests ont réussi.

Interprétation

Le test « intrusion » ne signifie pas qu’un objet inconnu sans certificat traverse mTLS. L’identité `capteur-test` a été volontairement créée pour vérifier la liste blanche du superviseur. Un objet non provisionné doit être rejeté dès la connexion TLS.

Voir aussi la capture Ubuntu transmise par Ilias, page suivante. Elle prouve la réception des mesures sur un second poste, pas l’exécution des tests de sécurité sur ce poste.

4. Rapport d’audit (extraits des preuves)

Résultats observés dans les fichiers de log

Connexion sans certificat et ACL :

```
[PREUVE mTLS] OK : le broker a coupé la connexion sans certificat client.

[PREUVE ACL] OK : publication interdite refusée, RC:135
```

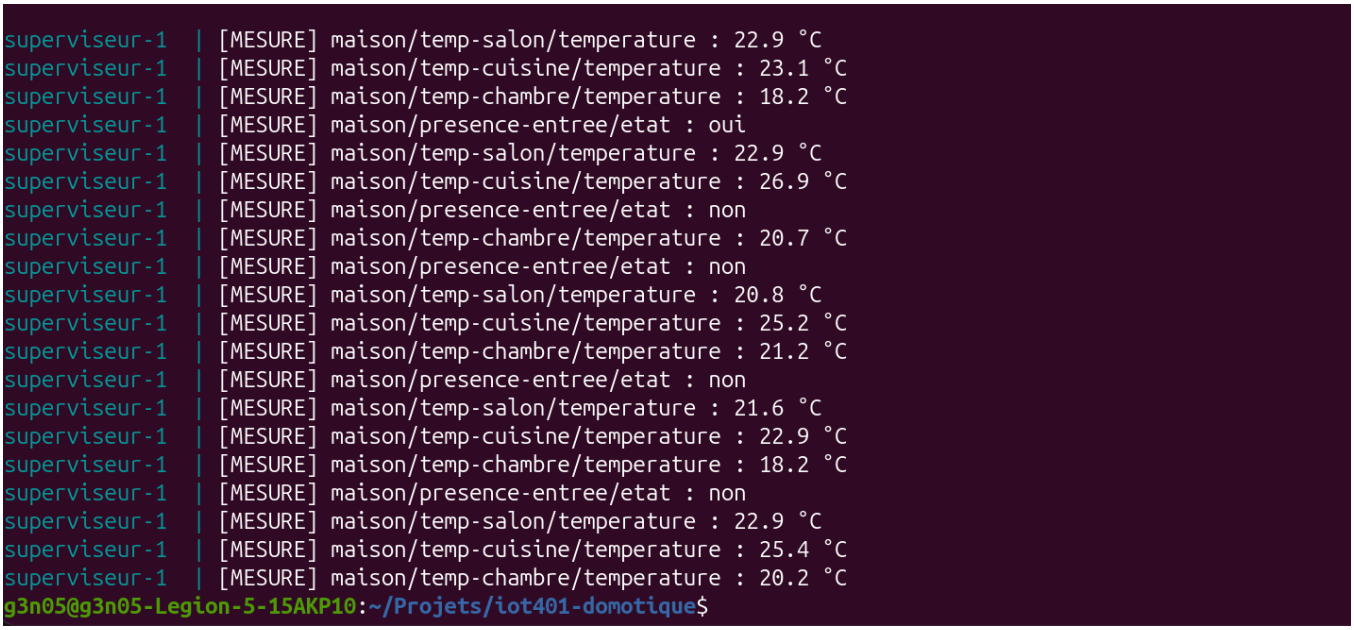
Alertes de supervision :

```
[ALERTE INTRUSION] Objet inconnu : topic=maison/capteur-test/etat

[ALERTE SEUIL] maison/temp-salon/temperature : 35.5 °C > 30.0 °C
```

Les fichiers bruts correspondants sont conservés dans preuves/. Les cinq tests unitaires Python ont aussi réussi. Les extractions ci-dessus reprennent les lignes constatées, sans résultat inventé.

Deuxième exécution : poste Ubuntu d’Ilias



Capture d’écran fournie par Ilias : le superviseur reçoit les mesures des trois températures et de la présence. La capture ne contient pas l’heure de la session et n’est donc pas utilisée pour dater les tests.

Réserve et piste de correction

Des avertissements sur les droits du fichier ACL de Mosquitto ont été constatés au démarrage initial. Ils n’ont pas empêché les essais, mais la gestion du propriétaire et des permissions doit être améliorée avant un déploiement réel. Cette correction est suivie dans l’issue GitHub n° 1.

5. Rapport de conformité

ETSI EN 303 645 - provisions visées

Thème ETSI	Positionnement et preuve
5.1 - Secrets d’accès / pas de mot de passe universel	Authentification par certificats uniques, pas de mot de passe partagé ; app/pki.py.
5.3 - Maintenir le logiciel à jour	Non implémenté : prévoir mises à jour testées et plan de support.
5.4 - Protection des paramètres sensibles	Clés en volumes individuels et droits restreints ; pas de stockage matériel sécurisé.
5.5 - Communications sécurisées	TLS mutualisé et contrôle du certificat serveur ; commun.py et config broker.
5.6 - Surface d’attaque minimale	Port MQTT non publié, réseau interne ; compose.yaml.
5.7 - Intégrité du logiciel	Secure boot / OTA signé non implémentés sur simulateurs.
5.8 - Données personnelles sécurisées	Présence booléenne, traitement local, ACL ; journalisation à mieux maîtriser.

OWASP IoT Top 10 (édition 2018) - risques pertinents

I1 identifiants faibles : évités via des certificats distincts ; **I2 services réseau non sécurisés :** broker non exposé ; **I4 mécanisme de mise à jour insuffisant :** reste à traiter ; **I5 composants obsolètes :** images/paquets versionnés, suivi de vulnérabilités nécessaire ; **I6 vie privée insuffisante :** données minimisées ; **I7 transferts/stockage non sécurisés :** TLS côté réseau, protection des volumes encore limitée ; **I8 gestion des objets :** inventaire statique, révocation automatique manquante.

Un « alignement » sur ces référentiels ne constitue pas une certification de conformité. Les chapitres ETSI sont des thèmes sélectionnés, pas un audit exhaustif de chaque provision.

5. Rapport de conformité (suite)

Cyber Resilience Act - règlement (UE) 2024/2847

Le CRA s'adresse aux produits comportant des éléments numériques mis à disposition sur le marché de l'UE. **Ce prototype purement pédagogique, non commercialisé, n'est pas présenté comme un produit certifié CRA.** Pour une commercialisation, le fabricant devrait documenter la gestion des vulnérabilités, les mises à jour de sécurité et la durée de support, l'analyse de risque, la documentation technique et la conformité exigée.

Au 28 octobre 2026, les obligations de déclaration de l'article 14 concernant certaines vulnérabilités activement exploitées et incidents graves sont applicables depuis le **11 septembre 2026** aux acteurs/produits dans le champ. L'application générale du CRA est prévue pour le **11 décembre 2027**. Ce prototype n'a ni processus de déclaration opérationnel ni maintien en conditions de sécurité contractuel.

RGPD : finalité, minimisation, conservation et droits

Principe	Décision domotique
Finalité	Alerter sur une température anormale et afficher l'occupation instantanée.
Minimisation	Une température par pièce, présence oui/non ; pas de visage, identité personnelle ni historique GPS.
Conservation effective	Pas de base, pas de retained MQTT ; des logs Docker restent tant que les conteneurs existent, limités par rotation en taille.
Amélioration de rétention	Politique de suppression programmée (**objectif : 24 h maximum pour les traces de présence**) et journalisation sans contenu personnel. Non implémentée ici.
Droits et information	Informer les occupants sur les capteurs et la finalité ; prévoir suppression des traces si nécessaire.

Dans une utilisation strictement personnelle ou domestique, une exemption RGPD peut s'appliquer à l'activité de la personne physique. Dès qu'un prestataire ou un service traite ces données pour son activité, l'analyse change. Nous adoptons néanmoins les principes de protection des données par conception.

6. Limites et améliorations

Prototype uniquement logiciel : pas de sondes physiques, d'autonomie batterie mesurée, de réseau Zigbee, de passerelle radio ni de table de bord graphique. Le débit et la portée ne sont pas validés.

Détection : les topics et les ACL identifient les sources attendues, mais le superviseur ne reçoit pas le CN dans le message MQTT lui-même. Un certificat de test volontairement délivré permet la preuve hors allowlist. Un véritable dispositif inconnu non provisionné est bloqué plus tôt par mTLS.

Cycle de vie : pas d'OTA signé, secure boot, révocation individuelle, quota de débit ou gestion automatique des expirations de certificat. Les clés restent lisibles par l'administrateur de l'hôte Docker. Les logs ont une rotation par taille et non une purge de 24 h garantie.

Améliorations : passerelle Zigbee physique, stockage des clés TPM/secure element, CRL/rotation par capteur, supervision de l'âge des données, audit central de sécurité, authentification renforcée des administrateurs, purge datée des journaux, CI de vérification et documentation d'exploitation.

7. Contributions du binôme

Répartition issue des commits GitHub et des essais constatés. À relire, corriger si besoin et signer par les deux membres avant dépôt : un commit ne suffit pas à prouver l'écriture originale de chaque ligne.

Activité	Benjamin TIBI	Ilias EL HOUZI
CDC, architecture et dossier	Rédaction initiale	Relecture à valider
Superviseur et tests Python	Code et tests	Vérification du fonctionnement
Capteurs, Mosquitto et MQTT	Intégration et essais	Ajout initial dans Git
PKI, ACL et Docker	Tests de sécurité	Configuration et correction Docker
Audit et preuves	Tests sur Ubuntu + logs	Capture Ubuntu du 2e PC

Signature Benjamin TIBI : _____ Date : _____

Signature Ilias EL HOUZI : _____ Date : _____

Sources utilisées

Énoncé IOT401 / barème / gabarit (28/10/2026) ; ETSI EN 303 645 ; OWASP IoT Top 10 2018 ; règlement UE 2024/2847 ; CNIL « minimiser les données collectées ». Liens complets disponibles dans le README du dépôt.